

Memory

Most of the time you do not think about memory in SPRFST. Values live where they need to and the runtime tidies up. This chapter is about what is actually happening, and the words you can use when you want control.

Where values live

Small values —`Int`, `Num`, `Bool`, `Byte` —live on the stack, inside the frame of the function using them. Nothing is allocated.

Bigger values —text, lists, maps, objects —live on the heap, and the variable holds a reference to them. Passing a list to a function passes the reference, not a copy, so it is cheap however long the list is.

```
use std.io

fn add_one(items: [Int]) {
    items.push(1)      ~~ the caller sees this
}

fn main() {
    var numbers = [0]
    add_one(numbers)
    io.say("{numbers}")
}
```

When you want a copy, ask for one:

```
use std.io

fn main() {
    let original = [1, 2, 3]
    var copy = clone(original)
    copy.push(4)
    io.say("original {original}")
    io.say("copy      {copy}")
}
```

Clearing up

The runtime collects values nothing refers to any more. Collection happens between statements, never in the middle of one, and it walks only live data. You can see the figures:

```
sprfst run main.spf --time
```

which reports instructions executed, peak memory and how many times the collector ran.

Ownership, when you want it

`own T` says a value has exactly one owner. Handing it to something else that wants to own it **moves** it, and the compiler stops you from using the old name afterwards.

```
use std.io

fn peek(items: ref [Int]) -> Int => items.len()
fn consume(items: own [Int]) -> Int => items.len()

fn main() {
    let a: own [Int] = [1, 2, 3]

    io.say("borrowing is fine: {peek(a)}")
    io.say("and `a` still works: {a.len()}")

    let b: own [Int] = a          ~~ ownership handed over
    io.say("`b` owns it now: {consume(b)}")

    let c: own [Int] = [9, 9]
    io.say("a copy keeps the original: {consume(clone(c))} and {c.len()}")
}
```

Use the moved name and the compiler says so, with the place it went:

```
error[E0301]  `a` was moved away
+- main.spf:6:22
|
6 |         io.say("{consume(a)}")
|                   - used after the move
5 |         let b: own [Int] = a
|                   - ownership moved here
|
+ note  `own` values have exactly one owner at a time
+ fixes
  • clone it first:  let copy = clone(a)
  • or borrow it instead:  fn use(x: ref List<Int>)
```

`ref T` borrows instead: the caller keeps ownership and the callee may read. `ref mut T` borrows for writing.

Ownership is opt in, and the check is deliberately simple—a name is treated as moved from the line the move appears on, so moving inside a branch counts for everything after it. Reach for `own` on large buffers, on handles that must be closed exactly once, and anywhere a second owner would be a bug. Leave it off and the collector handles everything, as it does in every other chapter.

Cleaning up at the right moment

`defer` runs a block when the function ends, however it ends—normally, through `give`, or through an error on the way out.

```

use std.io
use std.fs

fn work() {
    fs.write("scratch.txt", "hello")
    defer {
        fs.remove("scratch.txt")
        io.say("tidied up")
    }
    io.say("while we work the file exists: {fs.exists("scratch.txt")}")
}

fn main() {
    work()
    io.say("afterwards: {fs.exists("scratch.txt")}")
}

```

This is the tool to reach for when something must be released: a file, a lock, a socket, a database handle. An object may also declare a `drop` method, which the language reserves for the same purpose, but the runtime does not call it yet—today, `defer` is what actually runs, and the examples use it.

`unsafe`

An `unsafe` block lets you do things the checker cannot verify, such as talking to C. It is deliberately ugly so it is easy to find.

```

unsafe {
    ~~ raw pointer work goes here
}

```

Exercise

Write a function that takes a list, makes a sorted copy and returns it, leaving the original untouched. Prove it with a print before and after.